

Benchmark

Android Interview Preparation Guide

Voice-Controlled Alarm Timetable App - Kotlin + Jetpack Compose

1. Explain Your Project (30-second and detailed versions)
2. Project-Based Interview Q&A
3. Basic Android Interview Q&A (fundamentals)
4. Kotlin Interview Q&A
5. Keyword Glossary - Hilt, Retrofit and the whole Android/Kotlin ecosystem

1. Explain Your Project

The 30-second pitch (memorize this)

"Benchmark is a voice-controlled alarm timetable app I built in Kotlin with Jetpack Compose. You can literally speak to it - say *Add gym at 6 PM for one hour* and it creates the task in a Room database, schedules an exact alarm with AlarmManager, and at 6 PM it rings like a real alarm clock: full-screen over the lock screen, looping ringtone, vibration, with Stop, Snooze and Done buttons. The AI understanding comes from Google Gemini, but I also wrote an offline rule-based parser so voice commands work without internet. Login is Firebase Auth with email and Google Sign-In."

The detailed walkthrough

Problem it solves: Managing a daily schedule normally means lots of typing and taps. Benchmark makes it hands-free: add, move, delete, complete and star tasks by voice, and never miss one because reminders behave like real alarms, not quiet pings.

Architecture: Single-activity MVVM. Jetpack Compose renders the UI as a function of state. A TaskViewModel exposes the Room database as a StateFlow, so any change (from touch OR voice) automatically recomposes every screen. A ReminderScheduler object is the single source of truth for AlarmManager. BroadcastReceivers handle the alarm firing, the notification action buttons, and re-registering alarms after reboot.

Voice pipeline: SpeechRecognizer (speech-to-text) -> Gemini 1.5 Flash with a JSON-contract prompt that includes today's real schedule -> the returned action executes on the ViewModel -> TextToSpeech confirms and listens again. If the API fails or no key is configured, a local regex parser handles the same commands offline.

Alarm pipeline: Task row id becomes the PendingIntent request code (so alarms can be cancelled or replaced). The notification channel is deliberately silent because on Android 8+ the channel sound overrides setSound(); the per-task ringtone is played manually with MediaPlayer on the ALARM audio stream, looping, plus a repeating vibration pattern - exactly like an incoming call.

Screens: Dashboard (365-day calendar strip + chronological task timeline + smart add dialog), Timetable Grid (hour-by-hour with copy/paste), Focus (only starred tasks), Profile (real Firebase account, computed streak, animated Canvas pie chart, CSV export, sign out).

2. Project-Based Interview Q&A;

These are the questions an interviewer is most likely to ask AFTER seeing this project (like your CameraX question). Pattern for every answer: tool -> why over the alternative -> one gotcha you personally hit.

Alarms and background work

Q: What did you use to schedule reminders - and why not WorkManager?

A: AlarmManager with setExactAndAllowWhileIdle(). WorkManager is for deferrable work - the OS batches it to save battery and gives no timing guarantee. An alarm at 6:00 PM must fire at exactly 6:00 PM even in Doze mode, and that is exactly what AlarmManager exact alarms are for. Rule: user-facing clock time = AlarmManager; syncs/uploads = WorkManager.

Q: What changes on Android 12+ for exact alarms?

A: You need SCHEDULE_EXACT_ALARM (revocable by the user) or USE_EXACT_ALARM. I check alarmManager.canScheduleExactAlarms() and fall back to setWindow() - a one-minute window - instead of crashing with a SecurityException.

Q: What happens to your alarms when the phone reboots?

A: They are lost - AlarmManager does not persist across reboots. I registered a BroadcastReceiver for BOOT_COMPLETED that reads all pending tasks from Room and re-schedules them. It uses goAsync() plus a coroutine because a receiver cannot do database I/O on the main thread.

Q: How do you cancel one specific alarm?

A: The PendingIntent request code is the task's Room row id (insert returns the id as Long). Recreating the same PendingIntent with the same request code and calling alarmManager.cancel() removes it. All PendingIntents use FLAG_IMMUTABLE, mandatory since Android 12.

Notifications

Q: How did you play a different ringtone per task? setSound() should work, no?

A: No - on Android 8+ setSound() is ignored; the CHANNEL's sound always wins, and a channel's sound cannot be changed after creation. My solution: create one silent high-importance channel, then play the user's chosen ringtone manually with MediaPlayer using USAGE_ALARM audio attributes - it loops, follows the alarm volume slider, and stops via the notification's Stop/Snooze/Done action buttons.

Q: How does the reminder show over the lock screen like a WhatsApp call?

A: setFullScreenIntent() with the USE_FULL_SCREEN_INTENT permission, plus showWhenLocked/turnScreenOn on the activity. CATEGORY_ALARM + PRIORITY_MAX makes it a heads-up banner, and setOngoing(true) prevents swiping it away while it rings.

Q: Any real-device problems with notifications?

A: Yes - on a vivo Android 14 device the sound played but the notification never appeared, even with POST_NOTIFICATIONS granted. OEM skins (vivo/oppo/xiaomi) add their own notification gates and battery killers. I debugged with dumpsys audio and dumpsys notification because vivo suppresses logcat, bundled the small icon inside the app (OEMs can drop notifications using system icons), and users must enable Autostart + unrestricted battery.

Voice and AI

Q: Which API did you use for speech - a third-party SDK?

A: No SDK - Android's built-in SpeechRecognizer with a RecognitionListener for speech-to-text (RECORD_AUDIO permission), and TextToSpeech for spoken replies. The conversation loop is: TTS greets -> listen -> process -> TTS confirms -> listen again, so commands can be chained.

Q: How does Gemini turn speech into actions?

A: I send the transcript with a system prompt that forces a strict JSON contract, e.g. {action:'move', taskName:'Gym', time:'7:00 PM', reply:'...'}.

Crucially the prompt includes today's actual task list, so 'move gym to 7' resolves against real data. I parse the JSON and dispatch to the ViewModel.

Q: What if there is no internet or no API key?

A: A local rule-based regex parser handles the same commands offline - add, delete, move, complete, star, list. The API key lives in local.properties and is injected via BuildConfig, so it never enters git.

Data, state and auth

Q: Why Room instead of raw SQLite?

A: Compile-time verified SQL, entities as data classes, and reactive queries: my DAO returns Flow>, so every DB change automatically updates the UI with no manual refresh. Room is an abstraction over SQLite, processed at build time by KSP.

Q: StateFlow or LiveData - and why?

A: StateFlow. It is Kotlin-first, testable without Android, and readable outside the UI: my voice assistant calls tasks.value synchronously to find the task named 'gym'. I convert Room's cold Flow to hot state with stateIn(viewModelScope, Eagerly, emptyList()).

Q: Tell me about a hard bug in Firebase Google Sign-In.

A: The account picker opened, you picked an account, then nothing happened. Cause: Google validates the APK signing certificate SHA-1 against Firebase; my machine's debug keystore was not registered, and the failure surfaces as a silent RESULT_CANCELED with hidden status code 10 (DEVELOPER_ERROR). Fix: register the SHA-1 from keytool, and rewrite the handler to parse failures so configuration errors are never silent.

3. Basic Android Interview Q&A;

Fundamentals every Android interview covers, independent of your project.

Core components

Q: What are the four main components of an Android app?

A: Activity (one screen of UI), Service (background work without UI), BroadcastReceiver (responds to system/app events like BOOT_COMPLETED), ContentProvider (shares data between apps, e.g. Contacts). All must be declared in AndroidManifest.xml.

Q: Explain the Activity lifecycle.

A: onCreate (initialize UI) -> onStart (visible) -> onResume (interactive) -> onPause (partially hidden, save light state) -> onStop (fully hidden) -> onDestroy (finishing or config change). Rotation destroys and recreates the Activity - which is why ViewModel exists: it survives configuration changes.

Q: What is an Intent? Explicit vs implicit?

A: A messaging object to request an action. Explicit names the exact class (Intent(context, MainActivity::class.java)) - used inside your app. Implicit declares an action (ACTION_VIEW a URL) and the system finds an app to handle it. My ringtone picker uses an implicit intent; my alarm PendingIntents wrap explicit ones.

Q: What is a PendingIntent?

A: A token that lets ANOTHER process (AlarmManager, NotificationManager) execute an intent later as if your app sent it. Request code + intent identify it for cancel/update. FLAG_IMMUTABLE is required by default since Android 12.

Q: What is Context?

A: Handle to the app environment - resources, system services, starting components. Activity context is tied to the screen's lifecycle; applicationContext lives as long as the process. Using Activity context in long-lived objects causes memory leaks.

Q: What is an ANR?

A: Application Not Responding - the main thread blocked for about 5 seconds. Prevent it by moving I/O and heavy work off the main thread (coroutines with Dispatchers.IO, in my case Room suspend functions and viewModelScope).

UI and data

Q: Views (XML) vs Jetpack Compose?

A: XML is imperative: inflate a layout, then mutate views by hand. Compose is declarative: UI is a function of state, and when state changes the framework recomposes only what depends on it. Less code, no findViewById, no RecyclerView adapters - LazyColumn replaces all of it.

Q: What are remember, LaunchedEffect and DisposableEffect in Compose?

A: remember caches a value across recompositions; remember { mutableStateOf() } is local UI state. LaunchedEffect runs a coroutine when a key changes (I use it for the rotating AI hints). DisposableEffect runs cleanup when the composable leaves the screen (I release SpeechRecognizer/TTS there).

Q: How do you pass data between screens?

A: With Navigation Compose: route arguments for small values, or better, both screens observe the same ViewModel/StateFlow so there is a single source of truth. Between Activities: Intent extras (Parcelable for objects).

Q: Serializable vs Parcelable?

A: Both make objects transferable in Intents/Bundles. Serializable is Java reflection - slow, garbage-heavy. Parcelable is Android-specific and much faster; with @Parcelize in Kotlin it is one annotation.

Q: What are launch modes?

A: standard (new instance each time), singleTop (reuse if already on top - onNewIntent), singleTask (one instance per task, clears above it), singleInstance (alone in its own task). Notification taps commonly use FLAG_ACTIVITY_CLEAR_TOP or singleTop.

Q: How do runtime permissions work?

A: Declare in the manifest; dangerous ones (mic, notifications on 13+) must also be requested at runtime. Modern way: registerForActivityResult with RequestPermission/RequestMultiplePermissions contracts - which is what my MainActivity does for RECORD_AUDIO and POST_NOTIFICATIONS.

Q: APK vs AAB?

A: APK is the installable package. AAB (Android App Bundle) is what you upload to Play - Google generates optimized APKs per device (screen density, ABI, language), shrinking downloads.

4. Kotlin Interview Q&A;

Q: val vs var? And const?

A: val = read-only reference (assigned once), var = mutable. const val is a compile-time constant (top level or companion object, primitives/String only).

Q: How does Kotlin handle null safety?

A: Types are non-null by default; String? is nullable. Operators: ?. safe call, ?: elvis default, !! force (throws NPE - avoid), let for null-checked blocks. This kills most NullPointerExceptions at compile time.

Q: What is a data class?

A: A class for holding data - the compiler generates equals, hashCode, toString, copy and componentN. My Task and TaskEntity are data classes; copy(isCompleted = true) is how I toggle completion immutably.

Q: What is a sealed class and when did you use one?

A: A restricted hierarchy - all subclasses known at compile time, so when() is exhaustive without an else. My Screen navigation routes (Dashboard, Timetable, Focus, Profile) are a sealed class; the same pattern models UI states (Loading/Success/Error). Enums are similar but every value is a single constant; sealed subclasses can carry different data.

Q: object and companion object?

A: object = thread-safe singleton in one keyword - my ReminderScheduler and AlarmSoundPlayer.
companion object = static-like members inside a class (constants, factory methods), e.g. my channel id and action strings.

Q: lateinit vs lazy?

A: lateinit var: promise to initialize a mutable non-null var before use (crashes if not). by lazy { }: computes a val on first access, cached after - good for expensive objects.

Q: What are extension functions?

A: Add functions to existing types without inheritance. Example from my code: fun List<Task>.sortedByStartTime() parses '9:00 AM' style strings and sorts chronologically - called like a built-in method.

Q: Coroutines vs threads? What is suspend?

A: Coroutines are lightweight - thousands can run on a few threads; they suspend instead of blocking. suspend marks a function that can pause without blocking its thread (all my DAO methods). Structured concurrency: viewModelScope auto-cancels work when the ViewModel dies. Dispatchers.Main = UI, IO = disk/network, Default = CPU.

Q: Flow vs StateFlow vs SharedFlow?

A: Flow: cold async stream - starts per collector (Room queries). StateFlow: hot, always holds one current value - ideal for UI state, read synchronously with .value. SharedFlow: hot events without a current value (one-off messages like toasts).

Q: Scope functions - let, apply, run, also, with?

A: let: execute with it, returns lambda result - common for null checks. apply: configure this, returns the object - builders (my MediaPlayer().apply{...}). run: like let but with this. also: side effects, returns object. with: non-extension run.

5. Keyword Glossary - the Android/Kotlin ecosystem

One-liner definitions to recognize and define anything an interviewer names. Items marked [used in Benchmark] are in your project - be ready to go deeper on those.

Dependency Injection

Dependency Injection (DI) — A class receives its dependencies from outside instead of creating them - easier testing and swapping of implementations.

Hilt — Google's official DI library for Android, built on Dagger. Annotations like `@HiltAndroidApp`, `@AndroidEntryPoint`, `@Inject`, `@Module` generate the wiring at compile time. If asked 'would you add it?': yes - it would replace my manual creation of `GeminiManager` and the database with injected singletons.

Dagger 2 — The compile-time DI framework underneath Hilt - powerful but verbose; Hilt standardizes it for Android.

Koin — A lighter Kotlin DSL alternative to Hilt - runtime service locator, no code generation, simpler but errors surface at runtime instead of compile time.

Networking

Retrofit — Type-safe REST client from Square. You declare an interface with annotations (`@GET("users/{id}")`, `@POST`, `@Body`) and Retrofit generates the implementation; suspend functions give coroutine support. Not needed in Benchmark because the Gemini SDK handles its own networking - but this is how I would call any custom backend.

OkHttp — The HTTP engine under Retrofit (also from Square) - connection pooling, interceptors for logging/auth headers, caching, timeouts.

Gson / Moshi / kotlinx.serialization — JSON parsers: Gson (older, reflection), Moshi (modern, Kotlin-aware), `kotlinx.serialization` (official Kotlin, compile-time). They convert JSON to data classes - I parsed Gemini's JSON manually with `JSONObject` since it is one small object.

Ktor — JetBrains' Kotlin-native HTTP client/server framework - the multiplatform alternative to Retrofit.

Jetpack / Architecture

Jetpack — Google's suite of libraries (androidx.*) solving common Android problems with backward compatibility.

Jetpack Compose [used in Benchmark] — Declarative UI toolkit - UI is `@Composable` functions of state; changing state recomposes only affected parts. Requires the Compose compiler plugin (a dedicated Kotlin 2.0 plugin).

ViewModel [used in Benchmark] — Holds UI state, survives configuration changes (rotation), exposes state via `StateFlow/LiveData`; `viewModelScope` cancels coroutines automatically.

Room [used in Benchmark] — SQLite abstraction: `@Entity` tables, `@Dao` interfaces with verified SQL, reactive Flow queries, migrations between schema versions.

Navigation Compose [used in Benchmark] — Single-activity navigation: `NavHost` maps route strings to composables; `popUpTo` manages the back stack (how my login screen is removed after sign-in).

LiveData — Lifecycle-aware observable holder - the pre-Flow way to push data to UI; new code prefers `StateFlow`.

WorkManager — Guaranteed deferrable background work with constraints (charging, network) - survives reboots. For jobs, not clock-exact alarms.

AlarmManager [used in Benchmark] — System service for exact-time scheduling that can wake the device - the right tool for alarms and reminders.

DataStore — Modern replacement for SharedPreferences - coroutine/Flow based key-value (or typed Proto) storage without UI-thread blocking.

Paging 3 — Loads large lists in pages from DB/network as the user scrolls.

CameraX — Jetpack camera library - lifecycle-aware use cases (Preview, ImageCapture, ImageAnalysis) that hide device-specific Camera2 quirks. The 'manager' answer to your previous interview's camera question.

Media3 / ExoPlayer — Google's media playback library - streaming, DASH/HLS, custom players. My alarm uses raw MediaPlayer because a looping ringtone needs no streaming.

MVVM [used in Benchmark] — Model-View-ViewModel: View observes ViewModel state; ViewModel talks to data layer; survives config changes. MVP uses a Presenter with view interfaces (older); MVI adds a single immutable state + intents stream.

Clean Architecture — Layered separation - presentation / domain (use cases) / data - with dependencies pointing inward. Overkill for small apps, standard in large teams.

Repository pattern — A class that hides WHERE data comes from (network vs DB) and exposes one clean API to ViewModels.

Kotlin language & build

Coroutines [used in Benchmark] — Kotlin's lightweight concurrency - suspend functions, structured scopes, Dispatchers for thread pools.

Flow / StateFlow / SharedFlow [used in Benchmark] — Cold streams / hot state holder / hot event bus - the reactive backbone of modern Android.

KSP [used in Benchmark] — Kotlin Symbol Processing - runs annotation processors (Room, Hilt) at compile time; ~2x faster than KAPT, and its version must exactly match the Kotlin version.

KAPT — Older annotation processing via Java stubs - being replaced by KSP.

Gradle / Version Catalogs — Build system; build.gradle.kts declares plugins and dependencies in Kotlin. Version catalogs (libs.versions.toml) centralize versions.

BuildConfig [used in Benchmark] — Compile-time generated constants class - how my Gemini API key travels from local.properties into code without touching git.

ProGuard / R8 — Code shrinker + obfuscator for release builds - smaller APK, harder to reverse-engineer; needs keep rules for reflection-based libs.

KMP (Kotlin Multiplatform) — Share Kotlin business logic across Android/iOS/desktop; Compose Multiplatform shares UI too.

Firebase & services

Firebase Auth [used in Benchmark] — Managed sign-in: email/password, Google, phone. Google Sign-In validates the APK's SHA-1 certificate against the console.

Firestore / Realtime Database — Cloud NoSQL databases with live sync - my roadmap item for syncing tasks across devices.

FCM (Firebase Cloud Messaging) — Server-to-device push notifications - different from my locally scheduled AlarmManager notifications.

Crashlytics — Production crash reporting with stack traces and device breakdowns.

Google Play Services — On-device Google APIs (sign-in, maps, location) that apps call via client libraries.

Images, testing, misc

Glide / Coil — Image loading + caching. Coil is the Kotlin/Compose-native choice (AsyncImage); Glide is the Java-era standard.

Lottie — Renders After Effects JSON animations natively.

JUnit — Unit testing on the JVM - my streak calculator and voice parser are the obvious targets.

Espresso / Compose UI Test — On-device UI testing; Compose has its own test rule with `onNodeWithText`-style finders.

Mockito / MockK — Mocking libraries for isolating classes in tests - MockK is Kotlin-first.

LeakCanary — Detects memory leaks in debug builds automatically.

ADB [used while debugging] — Android Debug Bridge - install builds, capture screenshots, send test broadcasts, inspect system state with `dumpsys` (how I debugged the vivo notification issue when logcat was suppressed).

Final tip: for every 'what did you use for X' question, answer in three beats - the tool, why over the alternative, and one gotcha you hit on a real device. That third beat is what makes you sound like you built it, because you did.